

RAM

Name: George Safwat Wasfy Farag

Table of Contents:

Design of RAM without Bugs:	4
Assertions of RAM:	5
Configuration object of RAM:	6
Interface of RAM:	7
Top of RAM:	7
Test of RAM:	8
Environment of RAM:	10
Agent of RAM:	12
Sequence of RAM:	14
Sequence item of RAM:	18
Sequencer of RAM:	21
Coverage collector of RAM:	21
Driver of RAM:	24
Monitor of RAM:	25
Scoreboard of RAM:	27
Do file of RAM:	30
Simulation of RAM:	30
WAVE:	30
Assertions:	31
Functional coverage:	31
Transcripts:	32
Reset:	32
Write only	32
Read only:	32
Write and Read:	32
Report on RAM:	33
Statement coverage:	33
Branch coverage:	34
Condition coverage:	34
Toggle coverage:	35

Assertion coverage:	36
Cover groups:	37
Verification Plan:	37

Design of RAM without Bugs:

```
module RAM (ram_interface.DUT intf);

reg [7:0] MEM [255:0];
reg [7:0] Rd_Addr, Wr_Addr;
always @(posedge intf.clk) begin
    if (!intf.rst_n) begin
        intf.dout <= 0;
        intf.tx_valid <= 0;
        Rd_Addr <= 0;
        Wr_Addr <= 0;
    end
    else begin //put begin &
end
        if (intf.rx_valid) begin
            case (intf.din[9:8])
                2'b00 : Wr_Addr <= intf.din[7:0];
                2'b01 : MEM[Wr_Addr] <= intf.din[7:0];
                2'b10 : Rd_Addr <= intf.din[7:0];
                2'b11 : intf.dout <= MEM[Rd_Addr]; //bug
MEM[Rd_Addr] instead of MEM[Wr_Addr]
                default : intf.dout <= 0;
            endcase
        end
        intf.tx_valid <= (intf.din[9] && intf.din[8])? 1'b1 :
1'b0; //Deleting dependancy on rx_valid
    end
end
endmodule
```

Assertions of RAM:

```
module SVA(ram_interface.sva intf);
property p1;
@(posedge intf.clk) (!intf.rst_n) | => (
intf.tx_valid==1'b0 && intf.dout==8'b0); //Scheduling
semantics
endproperty
Assert_reset: assert property(p1);
cover_reset: cover property(p1);

property p2;
@(posedge intf.clk) disable iff(!intf.rst_n)
(intf.din[9:8]==2'b00 || intf.din[9:8]==2'b01 ||
intf.din[9:8]==2'b10) | => (intf.tx_valid==0);
endproperty
Assert_tx_valid: assert property(p2);
cover_tx_valid: cover property(p2);

property p3;
@(posedge intf.clk) disable iff(!intf.rst_n)
(intf.din[9:8]==2'b11) | => (intf.tx_valid==1) | =>
(intf.din[9:8]==2'b00 || intf.din[9:8]==2'b10) | =>
(intf.tx_valid==0) ;
endproperty
Assert_tx_valid_fell: assert property(p3);
cover_tx_valid_fell: cover property(p3);

property p4; //we did [=1:$] because 2'b00 can come more
than one time consecutively
```

```

@(posedge intf.clk) disable
iff(!intf.rst_n) (intf.din[9:8]==2'b00) [=1:$] ##1
(intf.din[9:8]==2'b01);
endproperty
Assert_wr_add_to_wr_data: assert property(p4);
cover_wr_add_to_wr_data:cover property(p4);

property p5;//we did [=1:$] because 2'b10 can come more
than one time consecutively
@(posedge intf.clk) disable
iff(!intf.rst_n) (intf.din[9:8]==2'b10) [=1:$]
##1 (intf.din[9:8]==2'b11);
endproperty
Assert_rd_add_to_rd_data: assert property(p5);
cover_rd_add_to_rd_data:cover property(p5);

endmodule

```

Configuration object of RAM:

```

package pkg_config;
import uvm_pkg::*;
`include "uvm_macros.svh"
class ram_config_obj extends uvm_object;
`uvm_object_utils(ram_config_obj)
virtual ram_interface ram_vif;
function new(string name="ram_config_obj");
super.new(name);
endfunction
endclass
endpackage

```

Interface of RAM:

```
interface ram_interface (clk);
//INPUTS
input bit clk;
bit rst_n, rx_valid;
bit [9:0] din;
//OUTPUTS
bit tx_valid; //Should be high if we want to read
logic [7:0] dout;

modport DUT( input clk, din, rst_n, rx_valid, output
tx_valid, dout);
modport sva(input clk, din, rst_n,
rx_valid, tx_valid, dout);
endinterface
```

Top of RAM:

```
import pkg_test::*;
import uvm_pkg::*;
`include "uvm_macros.svh"
module top();
bit clk,rst_n;

initial begin
    clk=0;
    forever #1 clk=~clk;
end
//instantiations
ram_interface intf(clk);
RAM DUT(intf);
```

```

SVA assertion(intf);
bind RAM SVA binding_all(intf);

initial begin
uvm_config_db #(virtual
ram_interface)::set(null,"uvm_test_top","ram_interface",
intf);
$readmemb("mem.dat", DUT.MEM );//initializing memory
run_test("ram_test");
end
endmodule

```

Test of RAM:

```

package pkg_test;
import agt_pkg::*;
import pkg_driver::*;
import uvm_pkg::*;
import seq_item_pkg::*;
import seq_pkg::*;
import sequencer_pkg::*;
import pkg_config::*;
import pkg_env::*;
`include "uvm_macros.svh"
class ram_test extends uvm_test;
`uvm_component_utils(ram_test)

virtual ram_interface ram_vif;
//build main sequence class & env class
ram_env env;
ram_config_obj ram_config;

```



```

reset_sequence reset_seq ;
write_only_sequence write_only_seq;
read_only_sequence read_only_seq;
write_read_sequence write_read_seq;
ram_agent agt;

//Provide the constructor default values
function new(string name="ram_test", uvm_component
parent =null) ;
super.new(name,parent);
endfunction

function void build_phase(uvm_phase phase);
super.build_phase(phase);

env=ram_env::type_id::create("env",this);
ram_config=
ram_config_obj::type_id::create("ram_config",this);
reset_seq= reset_sequence
::type_id::create("reset_seq",this);
write_only_seq= write_only_sequence
::type_id::create("write_only_seq",this);
read_only_seq=
read_only_sequence::type_id::create("read_only_seq",this
);
write_read_seq= write_read_sequence
::type_id::create("write_read_seq",this);

if(!uvm_config_db#(virtual ram_interface)::get
(this,"","ram_interface",ram_config.ram_vif))

```

```

`uvm_fatal("build_phase","Unable to get virtual
interface");

uvm_config_db
#(ram_config_obj)::set(this,"*", "CFG", ram_config);

endfunction

task run_phase(uvm_phase phase);
super.run_phase(phase);
phase.raise_objection(this);
`uvm_info("run_phase","stimulus generated
started",UVM_LOW);

reset_seq.start(env.agt.sqr);
write_only_seq.start(env.agt.sqr);
read_only_seq.start(env.agt.sqr);
write_read_seq.start(env.agt.sqr);

`uvm_info("run_phase","stimulus generated
ended",UVM_LOW);
phase.drop_objection(this);
endtask
endclass
endpackage

```

Environment of RAM:

```

package pkg_env;
import agt_pkg::*;
import scoreboard_pkg::*;

```

```

import cov_pkg::*;
import pkg_driver::*;
import uvm_pkg::*;
import seq_item_pkg::*;
import seq_pkg::*;
import sequencer_pkg::*;
`include "uvm_macros.svh"

class ram_env extends uvm_env;
    `uvm_component_utils(ram_env);

    ram_agent agt;
    ram_scoreboard sb;
    ram_coverage cov;

    function new(string name="ram_env",uvm_component parent
    =null) ;
        super.new(name,parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);

        agt=ram_agent::type_id::create("agt",this);
        sb=ram_scoreboard::type_id::create("sb",this);
        cov=ram_coverage::type_id::create("cov",this);
    endfunction

    function void connect_phase(uvm_phase phase);
        super.connect_phase(phase);
        agt.agt_ap.connect(sb.sb_export);
        agt.agt_ap.connect (cov.cov_export);

```

```
endfunction  
endclass  
endpackage
```

Agent of RAM:

```
package agt_pkg;  
`include "uvm_macros.svh"  
import uvm_pkg::*;  
import pkg_driver::*;  
import mon_pkg::*;  
import pkg_config::*;  
import sequencer_pkg::*;  
import seq_item_pkg::*;  
import seq_pkg::*;  
  
class ram_agent extends uvm_agent;  
  `uvm_component_utils(ram_agent);  
  
  ram_driver drv; //Create object for driver  
  ram_sequencer sqr; //Create object for sequencer  
  ram_monitor mon; //Create object for monitor  
  ram_config_obj ram_config; //Create object for  
  config_object  
  
  uvm_analysis_port #(ram_seq_item) agt_ap;  
  function new(string name="ram_agent", uvm_component  
    parent = null) ;  
    super.new(name, parent);  
  endfunction
```

```

//This function to Place it in the uvm hierarchy without
this function we cannot use this class
//you cannot make object without this function
//super.new ==> to go to parent uvm_agent and extends
from it and identify who will be parent to this agent
function void build_phase(uvm_phase phase);
super.build_phase(phase);

if(!uvm_config_db
#(ram_config_obj)::get(this,"","CFG",ram_config))
`uvm_fatal("build_phase","Unable to get configuration
object of driver");

drv=ram_driver::type_id:: create("drv",this);
sqr=ram_sequencer::type_id:: create("sqr",this);
mon=ram_monitor::type_id:: create("mon",this);
agt_ap=new("agt_ap",this);

endfunction

function void connect_phase(uvm_phase phase);
super.connect_phase(phase);
    //driver
    drv.ram_vif=ram_config.ram_vif;
    drv.seq_item_port.connect(sqr.seq_item_export);//TLM
    //monitor
    mon.ram_vif=ram_config.ram_vif;
    mon.mon_ap.connect(agt_ap);
endfunction

endclass
endpackage

```

Sequence of RAM:

```
package seq_pkg;
`include "uvm_macros.svh"
import uvm_pkg::*;
import seq_item_pkg::*;

class reset_sequence extends uvm_sequence
#(ram_seq_item);
`uvm_object_utils(reset_sequence)

ram_seq_item seq_item;

function new(string name="reset_sequence");
super.new(name);
endfunction

//Reset
task body();
seq_item=ram_seq_item::type_id::create("seq_item");//put
the object created in factory
$display("START OF RESET");
start_item(seq_item);

seq_item.din=0;
seq_item.rx_valid=0;
seq_item.rst_n=0;
finish_item(seq_item);
#1 $display("END OF RESET");
endtask
endclass
```

```

//WRITE ONLY
class write_only_sequence extends uvm_sequence
#(ram_seq_item);
`uvm_object_utils(write_only_sequence)

ram_seq_item seq_item;

function new(string name="write_only_sequence");
super.new(name);
endfunction

task body();
seq_item=ram_seq_item::type_id::create("seq_item");//put
the object created in factory
seq_item.read_only.constraint_mode(0);
seq_item.write_only.constraint_mode(1);
seq_item.mix_read_write.constraint_mode(0);
$display("START OF WRITE ONLY");
repeat(10000) begin
start_item(seq_item);

assert(seq_item.randomize());

finish_item(seq_item);
end
$display("END OF WRITE ONLY");
endtask
endclass

```

```

//READ ONLY
class read_only_sequence extends uvm_sequence
#(ram_seq_item);
`uvm_object_utils(read_only_sequence)

ram_seq_item seq_item;

function new(string name="read_only_sequence");
super.new(name);
endfunction

task body();
seq_item=ram_seq_item::type_id::create("seq_item");//put
the object created in factory
seq_item.read_only.constraint_mode(1);
seq_item.write_only.constraint_mode(0);
seq_item.mix_read_write.constraint_mode(0);
$display("START OF READ ONLY");
repeat(10000) begin
start_item(seq_item);

assert(seq_item.randomize());

finish_item(seq_item);
end
$display("END OF READ ONLY");
endtask
endclass

```



```

//WRITE & READ
class write_read_sequence extends uvm_sequence
#(ram_seq_item);
`uvm_object_utils(write_read_sequence)

ram_seq_item seq_item;

function new(string name="write_read_sequence");
super.new(name);
endfunction

task body();
seq_item=ram_seq_item::type_id::create("seq_item");//put
the object created in factory
seq_item.read_only.constraint_mode(0);
seq_item.write_only.constraint_mode(0);
seq_item.mix_read_write.constraint_mode(1);
$display("START OF WRITE & READ");
repeat(10000) begin

start_item(seq_item);

assert(seq_item.randomize());

finish_item(seq_item);
end
$display("END OF WRITE & READ");
Endtask
endclass
endpackage

```

Sequence item of RAM:

```
package seq_item_pkg;
`include "uvm_macros.svh"
import uvm_pkg::*;

class ram_seq_item extends uvm_sequence_item;
`uvm_object_utils(ram_seq_item)
rand bit rst_n,rx_valid;
rand bit [9:0] din;
bit tx_valid;
logic [7:0] dout;
bit [1:0] prev_din;

function new(string name="ram_seq_item");
super.new(name);
endfunction

function string convert2string();
return $sformatf("%s din=0b%b, rx_valid=0b%b ,rst_n=
0b%b, tx_valid= 0b%b ,dout= 0b%b",
super.convert2string(),din ,rx_valid ,rst_n ,
tx_valid , dout);
endfunction

function string convert2string_stimulus();
return $sformatf("%s din=0b%b, rx_valid=0b%b ,rst_n=
0b%b, tx_valid= 0b%b ,dout= 0b%b",
super.convert2string(),din ,rx_valid ,rst_n ,
tx_valid , dout);
endfunction
```

```

//CONSTRAINTS

constraint ram{
    rst_n dist{1:/95, 0:/5};
    rx_valid dist{1:/95, 0:/5};
}

constraint write_only{
    (prev_din==2'b00)->{din[9:8] dist{2'b00:/30,
2'b01:/70}};
    (prev_din==2'b01)->{din[9:8] dist{2'b01:/30,
2'b00:/70}};
}

constraint read_only{
    (prev_din==2'b10)->{din[9:8]==2'b11};
    (prev_din==2'b11)->{din[9:8]==2'b10};
}

constraint mix_read_write{
    (prev_din==2'b00) -> {din[9:8] dist{2'b00:/50,
2'b01:/50}};
    //After a Write Data 60% → Read Address & 40% →
Write Address
    (prev_din==2'b01) -> {din[9:8] dist{2'b10:/60,
2'b00:/40}};
    //Read address
    (prev_din==2'b10) -> {din[9:8] == 2'b11};
    //After a Read Data 60% → Write Address & 40% →
Read Address

```

```
        (prev_din==2'b11) -> {din[9:8] dist{2'b00:/60,  
2'b10:/40}};  
    }  
function void post_randomize();  
prev_din=din[9:8]; //To check the prev_din with the din  
now in constraints  
endfunction  
  
endclass  
endpackage
```

Sequencer of RAM:

```
package sequencer_pkg;
`include "uvm_macros.svh"
import seq_item_pkg::*;
import uvm_pkg::*;
class ram_sequencer extends uvm_sequencer
#(ram_seq_item);
`uvm_component_utils(ram_sequencer);

function new(string name="ram_sequencer",uvm_component
parent=null);
super.new(name,parent);
endfunction
endclass
endpackage
```

Coverage collector of RAM:

```
package cov_pkg;
import uvm_pkg::*;
import seq_item_pkg::*;
import pkg_config::*;
`include "uvm_macros.svh"
class ram_coverage extends uvm_component;
`uvm_component_utils(ram_coverage)

virtual ram_interface ram_vif;
uvm_analysis_export #(ram_seq_item) cov_export;
uvm_tlm_analysis_fifo #(ram_seq_item) cov_fifo;
ram_seq_item seq_item_cov;
```

```

covergroup cvr_gp;

Transaction_ordering_cp: coverpoint
seq_item_cov.din[9:8]
{
    bins possible_values={2'b00,2'b01,2'b10,2'b11};
    bins write_address_then_write_data=(2'b00 => 2'b01);
    bins read_address_then_read_data=(2'b10 => 2'b11);
    bins
wr_address_then_wr_data_rd_address_then_rd_data=
(2'b00=>2'b01=>2'b10=>2'b11);
}
rx_valid_high_cp: coverpoint seq_item_cov.rx_valid
{bins rx_valid_high={1};}
tx_valid_high_cp: coverpoint seq_item_cov.tx_valid
iff(seq_item_cov.din[9:8]==2'b11)
{bins tx_valid_high={1};}

rx_valid_with_all_bins_din: cross
Transaction_ordering_cp,rx_valid_high_cp;
rd_data_with_tx_valid_high: cross
Transaction_ordering_cp,tx_valid_high_cp
{ignore_bins
write_address_then_write_data_with_tx_valid_high=
binsof(Transaction_ordering_cp.write_address_then_write_
data);}

endgroup

function new(string name="alsu_scoreboard",
uvm_component parent =null) ;
super.new(name,parent);

```

```

cvr_gp=new();//create covergroup
endfunction

function void build_phase(uvm_phase phase);
super.build_phase(phase);
cov_export=new("cov_export",this);
cov_fifo=new("cov_fifo",this);
endfunction

function void connect_phase(uvm_phase phase);
super.connect_phase(phase);
cov_export.connect(cov_fifo.analysis_export);
endfunction

task run_phase(uvm_phase phase);
super.run_phase(phase);
forever begin
cov_fifo.get(seq_item_cov);
cvr_gp.sample();
end
endtask

endclass
endpackage

```

Driver of RAM:

```
package pkg_driver;
import uvm_pkg::*;
import pkg_config::*;
import sequencer_pkg::*;
import seq_item_pkg::*;
import seq_pkg::*;
`include "uvm_macros.svh"
class ram_driver extends uvm_driver #(ram_seq_item);
`uvm_component_utils(ram_driver)

virtual ram_interface ram_vif;
ram_seq_item stim_seq_item;

function new(string name="ram_driver",uvm_component
parent =null) ;
super.new(name,parent);
endfunction

task run_phase(uvm_phase phase);
forever begin
    stim_seq_item=ram_seq_item::type_id::create("stim_seq_item");
    seq_item_port.get_next_item(stim_seq_item); //start of pulling data from sequencer

    ram_vif.rst_n=stim_seq_item.rst_n;
    ram_vif.din=stim_seq_item.din;
    ram_vif.rx_valid=stim_seq_item.rx_valid;
    @(negedge ram_vif.clk);
```



```

        seq_item_port.item_done(); //End of
pilling
        `uvm_info("run_phase",stim_seq_item.convert2string_
stimulus(),UVM_HIGH);
end
endtask
endclass
endpackage

```

Monitor of RAM:

```

package mon_pkg;
`include "uvm_macros.svh"
import uvm_pkg::*;
import pkg_config::*;
import sequencer_pkg::*;
import seq_item_pkg::*;
class ram_monitor extends uvm_monitor;
`uvm_component_utils(ram_monitor)

virtual ram_interface ram_vif ;
ram_seq_item rsp_seq_item;
uvm_analysis_port #(ram_seq_item) mon_ap;

function new(string name="ram_monitor",uvm_component
parent =null) ;
super.new(name,parent);
endfunction

function void build_phase(uvm_phase phase);
    super.build_phase(phase);

```

```

        mon_ap = new("mon_ap", this);
endfunction

task run_phase(uvm_phase phase);
super.run_phase(phase);
forever begin
    rsp_seq_item=ram_seq_item::type_id::
create("rsp_seq_item",this);
@(negedge ram_vif.clk)

    rsp_seq_item.rst_n=ram_vif.rst_n;
    rsp_seq_item.din=ram_vif.din;
    rsp_seq_item.rx_valid=ram_vif.rx_valid;
    rsp_seq_item.tx_valid=ram_vif.tx_valid;
    rsp_seq_item.dout=ram_vif.dout;

    mon_ap.write(rsp_seq_item);
    `uvm_info("run_phase",rsp_seq_item.convert2string(),
UVM_HIGH);
end

endtask
endclass
endpackage

```

Scoreboard of RAM:

```
package scoreboard_pkg;
`include "uvm_macros.svh"
import uvm_pkg::*;
import seq_item_pkg::*;
import seq_pkg::*;
class ram_scoreboard extends uvm_scoreboard;
`uvm_component_utils(ram_scoreboard)

uvm_analysis_export #(ram_seq_item) sb_export;
uvm_tlm_analysis_fifo #(ram_seq_item) sb_fifo;
ram_seq_item seq_item_sb;

int error_count=0;
int correct_count=0;
logic tx_valid_ref;
logic [7:0] dout_ref;

reg [7:0] MEM [255:0];
reg [7:0] Rd_Addr, Wr_Addr;

function new(string name="ram_scoreboard",uvm_component
parent=null);
super.new(name,parent);
endfunction

//BUILD ANALYSIS EXPORT AND FIFO
function void build_phase(uvm_phase phase);
super.build_phase(phase);
sb_export=new("sb_export",this);
sb_fifo=new("sb_fifo",this);
```

```

endfunction

//CONNECT ANALYSIS EXPORT
function void connect_phase(uvm_phase phase);
super.connect_phase(phase);
sb_export.connect(sb_fifo.analysis_export);//Connecting
fifo with scoreboard
endfunction

task run_phase(uvm_phase phase);
super.run_phase(phase);
forever begin
sb_fifo.get(seq_item_sb);
ref_model(seq_item_sb);
if((seq_item_sb.dout!=dout_ref)&&(seq_item_sb.tx_valid!=
tx_valid_ref))begin
    `uvm_error("run_phase",$sformatf("comparison
failed,transaction recieved by the
DUT:%s",seq_item_sb.convert2string()));
        error_count++;
    end
    else begin
        `uvm_info("run_phase",$sformatf("correct out_refput:%s
",seq_item_sb.convert2string()),UVM_LOW);
            correct_count++;
        end
    end
end

endtask

task ref_model(ram_seq_item seq_item_chk);
if (!seq_item_chk.rst_n) begin
    dout_ref = 0;

```

```

        tx_valid_ref = 0;
        Rd_Addr = 0;
        Wr_Addr = 0;
    end
    else begin
        if (seq_item_chk.rx_valid) begin
            if (seq_item_chk.din[9:8]== 2'b00)
                Wr_Addr = seq_item_chk.din[7:0];
            else if (seq_item_chk.din[9:8]== 2'b01)
                MEM[Wr_Addr] = seq_item_chk.din[7:0];
            else if (seq_item_chk.din[9:8]== 2'b10)
                Rd_Addr = seq_item_chk.din[7:0];
            else if (seq_item_chk.din[9:8]== 2'b11)
                dout_ref = MEM[Wr_Addr];
            else dout_ref = 0;
        end
        tx_valid_ref = (seq_item_chk.din[9] &&
seq_item_chk.din[8])? 1'b1 : 1'b0;
    end
endtask

function void report_phase (uvm_phase phase);
    super.report_phase(phase);
    `uvm_info("report_phase",$sformatf("Total
successful counts:%0d",correct_count),UVM_MEDIUM);
    `uvm_info("report_phase",$sformatf("Total failed
counts:%0d",error_count),UVM_MEDIUM);
endfunction
endclass
endpackage

```

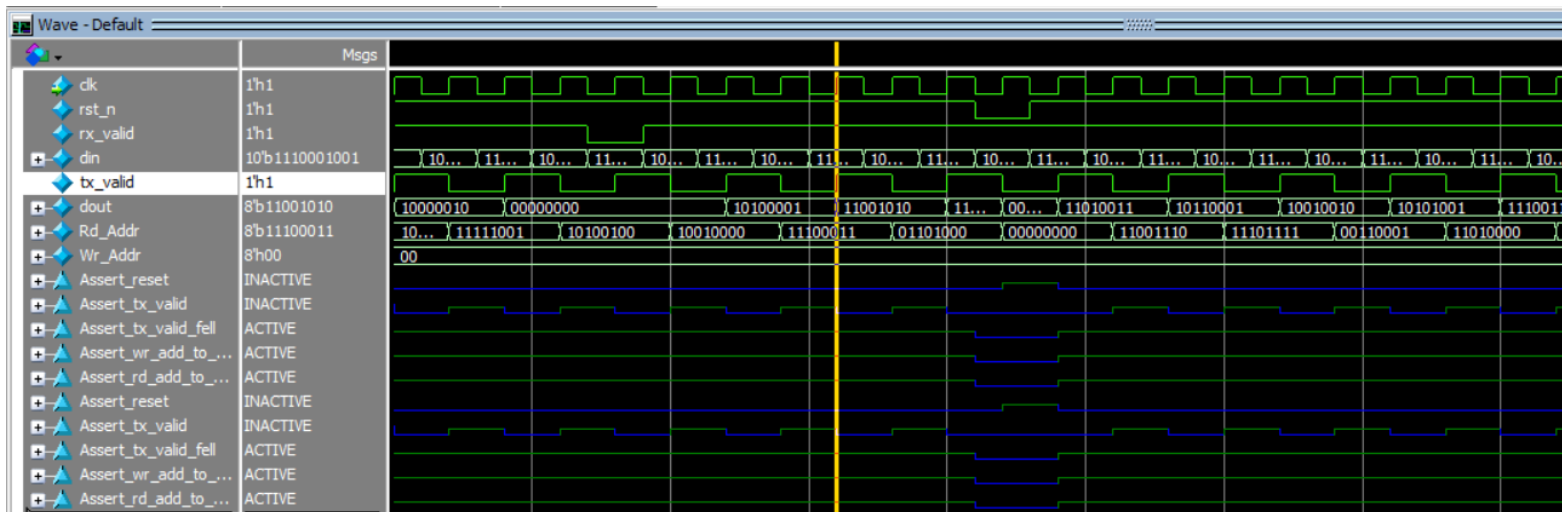
Do file of RAM:

```
vlib work
vlog -f ram/src_files.list\
      +incdir+ram/uvm\
      +incdir+ram/rtl +cover -covercells

vsim -voptargs+=acc work.top -cover -classdebug -uvmcontrol=all
coverage exclude -src ram/rtl/RAM.sv -line 20 -code b
coverage exclude -src ram/rtl/RAM.sv -line 20 -code s
coverage save ram.ucdb -onexit -du work.RAM
add wave -position insertpoint sim:/top/intf/*
add wave -position insertpoint sim:/top/DUT/*
add wave /top/DUT/binding_all/Assert_reset /top/DUT/binding_all/Assert_tx_valid /top/DUT/binding_all/Assert_tx_valid_fell /top/DUT/binding_all/Assert_wr_add_to_wr_data
/top/DUT/binding_all/Assert_rd_add_to_rd_data /top/assertion/Assert_reset /top/assertion/Assert_tx_valid /top/assertion/Assert_tx_valid_fell
/top/assertion/Assert_wr_add_to_wr_data /top/assertion/Assert_rd_add_to_rd_data
run -all
coverage report -detail -cvg -directive -comments -file F_cover_RAM.txt -noa -all
quit -sim
vcover report ram.ucdb -details -all -output coverage_rpt_RAM.txt
```

Simulation of RAM:

WAVE:



Assertions:

▲ /seq_pkg::write_only_sequence::body/#...	Immediate	SVA	on	0	1	-	-	-	-	off	assert (randomize(...))	✓
▲ /seq_pkg::read_only_sequence::body/#...	Immediate	SVA	on	0	1	-	-	-	-	off	assert (randomize(...))	✓
▲ /seq_pkg::write_read_sequence::body/...	Immediate	SVA	on	0	1	-	-	-	-	off	assert (randomize(...))	✓
▲ /top/DUT/binding_all/Assert_reset	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0 off	assert(@(posedge intf.clk) (~intf.rst...	✓
▲ /top/DUT/binding_all/Assert_tx_valid	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0 off	assert(@(posedge intf.clk) disable if...	✓
▲ /top/DUT/binding_all/Assert_tx_valid_fell	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0 off	assert(@(posedge intf.clk) disable if...	✓
▲ /top/DUT/binding_all/Assert_wr_add_to_...	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0 off	assert(@(posedge intf.clk) disable if...	✓
▲ /top/DUT/binding_all/Assert_rd_add_to_...	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0 off	assert(@(posedge intf.clk) (~intf.rst...	✓
▲ /top/assertion/Assert_reset	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0 off	assert(@(posedge intf.clk) disable if...	✓
▲ /top/assertion/Assert_tx_valid	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0 off	assert(@(posedge intf.clk) disable if...	✓
▲ /top/assertion/Assert_tx_valid_fell	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0 off	assert(@(posedge intf.clk) disable if...	✓
▲ /top/assertion/Assert_wr_add_to_wr_da...	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0 off	assert(@(posedge intf.clk) disable if...	✓
▲ /top/assertion/Assert_rd_add_to_rd_dat...	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0 off	assert(@(posedge intf.clk) disable if...	✓

Functional coverage:

Name	Class Type	Coverage	Goal	% of Goal	Status	Included	Merge_instances	Get_inst_coverage	Comment
/cov_pkg/ram_coverage		100.0%							
TYPE cvr_gp		100.0%	100	100.0%	✓		auto(0)		
CVP cvr_gp::Transaction_ordering_cp	ram_coverage	100.0%	100	100.0%	✓				
CVP cvr_gp::rx_valid_high_cp	ram_coverage	100.0%	100	100.0%	✓				
CVP cvr_gp::tx_valid_high_cp	ram_coverage	100.0%	100	100.0%	✓				
CROSS cvr_gp::rx_valid_with_all_bins_din	ram_coverage	100.0%	100	100.0%	✓				
CROSS cvr_gp::rd_data_with_tx_valid_high	ram_coverage	100.0%	100	100.0%	✓				
INST \cov_pkg::ram_coverage::cvr_gp		100.0%	100	100.0%	✓				0
CVP Transaction_ordering_cp		100.0%	100	100.0%	✓				
bin possible_values		30001	1	100.0%	✓				
bin write_address_then_write_data		5735	1	100.0%	✓				
bin read_address_then_read_data		5734	1	100.0%	✓				
bin wr_address_then_wr_data_rd_address_then_rd_data		1047	1	100.0%	✓				
CVP rx_valid_high_cp		100.0%	100	100.0%	✓				
bin rx_valid_high		28471	1	100.0%	✓				
CVP tx_valid_high_cp		100.0%	100	100.0%	✓				
bin tx_valid_high		6807	1	100.0%	✓				
CROSS rx_valid_with_all_bins_din		100.0%	100	100.0%	✓				
bin <possible_values,rx_valid_high>		28471	1	100.0%	✓				
bin <write_address_then_write_data,rx_valid_high>		5453	1	100.0%	✓				
bin <read_address_then_read_data,rx_valid_high>		5448	1	100.0%	✓				
bin <wr_address_then_wr_data_rd_address_then_rd_data,rx_valid_high>		994	1	100.0%	✓				
CROSS rd_data_with_tx_valid_high		100.0%	100	100.0%	✓				
bin <possible_values,tx_valid_high>		6807	1	100.0%	✓				
bin <read_address_then_read_data,tx_valid_high>		5452	1	100.0%	✓				
bin <wr_address_then_wr_data_rd_address_then_rd_data,tx_valid_high>		997	1	100.0%	✓				
ignore_bin write_address_then_write_data_with_tx_valid_high		0	-	-	✓				

Report on RAM:

Statement coverage:

Coverage Report by file with details

=== File: ram/rtl/RAM.sv

Statement Coverage:

Enabled Coverage	Active	Hits	Misses	% Covered
Stmts	10	10	0	100.0

=====Statement Details=====

Statement Coverage for file ram/rtl/RAM.sv --

```
1      module RAM (ram_interface.DUT intf);
2
3      reg [7:0] MEM [255:0];
4      reg [7:0] Rd_Addr, Wr_Addr;
5
6      1      30001  always @(posedge intf.clk) begin
7
8      1      1482      if (!intf.rst_n) begin
9      1      1482          intf.dout <= 0;
10     1      1482          intf.tx_valid <= 0;
11     1      1482          Rd_Addr <= 0;
12     1      1482          Wr_Addr <= 0;
13     end
14     else begin //put begin & end
15         if (intf.rx_valid) begin
16             case (intf.din[9:8])
17             1      6996         2'b00 : Wr_Addr <= intf.din[7:0];
18             1      6553         2'b01 : MEM[Wr_Addr] <= intf.din[7:0];
19             1      7033         2'b10 : Rd_Addr <= intf.din[7:0];
20             1      6477         2'b11 : intf.dout <= MEM[Rd_Addr]; //bug MEM[Rd_Addr] instead of MEM[Wr_Addr]
21             default : intf.dout <= 0;
22         endcase
23     end
24     1      28519  intf.tx_valid <= (intf.din[9] && intf.din[8])? 1'b1 : 1'b0; //Deleting dependancy on rx_valid
25     end
26
27     endmodule
```

Branch coverage:

```
Branch Coverage:
Enabled Coverage      Active      Hits      Misses % Covered
-----
Branches              8          8          0    100.0

=====Branch Details=====

Branch Coverage for file ram/rtl/RAM.sv --

-----IF Branch-----
7          1          30001      Count coming in to IF
7          1          1482       if (intf.rst_n) begin
13         1          28519      else begin //put begin & end
Branch totals: 2 hits of 2 branches = 100.0%

-----IF Branch-----
14         1          28519      Count coming in to IF
14         1          27059      if (intf.rx_valid) begin
All False Count 1460
Branch totals: 2 hits of 2 branches = 100.0%

-----CASE Branch-----
15         1          27059      Count coming in to CASE
16         1          6996       2'b00 : Wr_Addr <= intf.din[7:0];
17         1          6553       2'b01 : MEM[Wr_Addr] <= intf.din[7:0];
18         1          7033       2'b10 : Rd_Addr <= intf.din[7:0];
19         1          6477       2'b11 : intf.dout <= MEM[Rd_Addr]; //bug MEM[Rd_Addr] instead of MEM[Wr_Addr]
20         1          E          default : intf.dout <= 0;
Branch totals: 4 hits of 4 branches = 100.0%
```

Condition coverage:

```
Condition Coverage:
Enabled Coverage      Active      Covered      Misses % Covered
-----
FEC Condition Terms      0          0          0    100.0

Expression Coverage:
Enabled Coverage      Active      Covered      Misses % Covered
-----
FEC Expression Terms      2          2          0    100.0

=====Expression Details=====

Expression Coverage for file ram/rtl/RAM.sv --

-----Focused Expression View-----
Line      23 Item      1 (intf.din[9] && intf.din[8])
Expression totals: 2 of 2 input terms covered = 100.0%

Input Term      Covered      Reason for no coverage      Hint
-----
intf.din[9]      Y
intf.din[8]      Y

Rows:      Hits      FEC Target      Non-masking condition(s)
-----
Row 1:      1      intf.din[9]_0      -
Row 2:      1      intf.din[9]_1      intf.din[8]
Row 3:      1      intf.din[8]_0      intf.din[9]
Row 4:      1      intf.din[8]_1      intf.din[9]
```

Toggle coverage:

FSM Coverage:

Enabled Coverage	Active	Hits	Misses	% Covered
-----	-----	----	-----	-----
FSMs				100.0
States	0	0	0	100.0
Transitions	0	0	0	100.0

Toggle Coverage:

Enabled Coverage	Active	Hits	Misses	% Covered
-----	-----	----	-----	-----
Toggle Bins	32	32	0	100.0

=====Toggle Details=====

Toggle Coverage for File ram/rtl/RAM.sv --

Line	Node	1H->0L	0L->1H	"Coverage"
-----	-----	-----	-----	-----
4	<u>Wr_Addr[7]</u>	1	1	100.00
4	<u>Wr_Addr[6]</u>	1	1	100.00
4	<u>Wr_Addr[5]</u>	1	1	100.00
4	<u>Wr_Addr[4]</u>	1	1	100.00
4	<u>Wr_Addr[3]</u>	1	1	100.00
4	<u>Wr_Addr[2]</u>	1	1	100.00
4	<u>Wr_Addr[1]</u>	1	1	100.00
4	<u>Wr_Addr[0]</u>	1	1	100.00
4	<u>Rd_Addr[7]</u>	1	1	100.00
4	<u>Rd_Addr[6]</u>	1	1	100.00
4	<u>Rd_Addr[5]</u>	1	1	100.00
4	<u>Rd_Addr[4]</u>	1	1	100.00
4	<u>Rd_Addr[3]</u>	1	1	100.00
4	<u>Rd_Addr[2]</u>	1	1	100.00
4	<u>Rd_Addr[1]</u>	1	1	100.00
4	<u>Rd_Addr[0]</u>	1	1	100.00

Total Node Count = 16

Toggled Node Count = 16

Untoggled Node Count = 0

Toggle Coverage = 100.0% (32 of 32 bins)

Assertion coverage:

DIRECTIVE COVERAGE:

Name	Design Unit	Design UnitType	Lang	File(Line)	Count	Status
/\top#DUT /binding_all/cover_reset	SVA	Verilog	SVA	ram/rtl/Assertions.sv(6)	1482	Covered
/\top#DUT /binding_all/cover_tx_valid	SVA	Verilog	SVA	ram/rtl/Assertions.sv(13)	20627	Covered
/\top#DUT /binding_all/cover_tx_valid_fell	SVA	Verilog	SVA	ram/rtl/Assertions.sv(19)	2052	Covered
/\top#DUT /binding_all/cover_wr_add_to_wr_data	SVA	Verilog	SVA	ram/rtl/Assertions.sv(25)	16620	Covered
/\top#DUT /binding_all/cover_rd_add_to_rd_data	SVA	Verilog	SVA	ram/rtl/Assertions.sv(31)	16457	Covered

TOTAL DIRECTIVE COVERAGE: 100.0% COVERS: 5

ASSERTION RESULTS:

Name	File(Line)	Failure Count	Pass Count
/\top#DUT /binding_all/Assert_reset	ram/rtl/Assertions.sv(5)	0	1
/\top#DUT /binding_all/Assert_tx_valid	ram/rtl/Assertions.sv(12)	0	1
/\top#DUT /binding_all/Assert_tx_valid_fell	ram/rtl/Assertions.sv(18)	0	1
/\top#DUT /binding_all/Assert_wr_add_to_wr_data	ram/rtl/Assertions.sv(24)	0	1
/\top#DUT /binding_all/Assert_rd_add_to_rd_data	ram/rtl/Assertions.sv(30)	0	1

Total Coverage By File (code coverage only, filtered view): 100.0%

DIRECTIVE COVERAGE:

Name	Design Unit	Design UnitType	Lang	File(Line)	Count	Status
/top/DUT/binding_all/cover_reset	SVA	Verilog	SVA	ram/rtl/Assertions.sv(6)	1482	Covered
/top/DUT/binding_all/cover_tx_valid	SVA	Verilog	SVA	ram/rtl/Assertions.sv(13)	20627	Covered
/top/DUT/binding_all/cover_tx_valid_fell	SVA	Verilog	SVA	ram/rtl/Assertions.sv(19)	2052	Covered
/top/DUT/binding_all/cover_wr_add_to_wr_data	SVA	Verilog	SVA	ram/rtl/Assertions.sv(25)	16620	Covered
/top/DUT/binding_all/cover_rd_add_to_rd_data	SVA	Verilog	SVA	ram/rtl/Assertions.sv(31)	16457	Covered
/top/assertion/cover_reset	SVA	Verilog	SVA	ram/rtl/Assertions.sv(6)	1482	Covered
/top/assertion/cover_tx_valid	SVA	Verilog	SVA	ram/rtl/Assertions.sv(13)	20627	Covered
/top/assertion/cover_tx_valid_fell	SVA	Verilog	SVA	ram/rtl/Assertions.sv(19)	2052	Covered
/top/assertion/cover_wr_add_to_wr_data	SVA	Verilog	SVA	ram/rtl/Assertions.sv(25)	16620	Covered
/top/assertion/cover_rd_add_to_rd_data	SVA	Verilog	SVA	ram/rtl/Assertions.sv(31)	16457	Covered

TOTAL DIRECTIVE COVERAGE: 100.0% COVERS: 10

Cover groups:

COVERGROUP COVERAGE:			
Covergroup	Metric	Goal	Status

TYPE /cov_pkg/nam_coverage/cvr_gp	100.0%	100	Covered
covered/total bins:	13	13	
missing/total bins:	0	13	
% Hit:	100.0%	100	
Coverpoint cvr_gp::Transaction_ordering_cp	100.0%	100	Covered
covered/total bins:	4	4	
missing/total bins:	0	4	
% Hit:	100.0%	100	
Coverpoint cvr_gp::rx_valid_high_cp	100.0%	100	Covered
covered/total bins:	1	1	
missing/total bins:	0	1	
% Hit:	100.0%	100	
Coverpoint cvr_gp::tx_valid_high_cp	100.0%	100	Covered
covered/total bins:	1	1	
missing/total bins:	0	1	
% Hit:	100.0%	100	
Cross cvr_gp::rx_valid_with_all_bins_din	100.0%	100	Covered
covered/total bins:	4	4	
missing/total bins:	0	4	
% Hit:	100.0%	100	
Cross cvr_gp::rd_data_with_tx_valid_high	100.0%	100	Covered
covered/total bins:	3	3	
missing/total bins:	0	3	
% Hit:	100.0%	100	
CLASS nam_coverage			
Covergroup instance \cov_pkg::nam_coverage::cvr_gp	100.0%	100	Covered
covered/total bins:	13	13	
missing/total bins:	0	13	
% Hit:	100.0%	100	
Coverpoint Transaction_ordering_cp	100.0%	100	Covered
covered/total bins:	4	4	
missing/total bins:	0	4	
% Hit:	100.0%	100	
bin possible_values	30001	1	Covered
bin write_address_then_write_data	5735	1	Covered
bin read_address_then_read_data	5734	1	Covered
bin wn_address_then_wn_data_rd_address_then_rd_data	1047	1	Covered
Coverpoint rx_valid_high_cp	100.0%	100	Covered
covered/total bins:	1	1	
missing/total bins:	0	1	
% Hit:	100.0%	100	
bin rx_valid_high	28471	1	Covered
Coverpoint tx_valid_high_cp	100.0%	100	Covered
covered/total bins:	1	1	
missing/total bins:	0	1	
% Hit:	100.0%	100	
bin tx_valid_high	6807	1	Covered
Cross rx_valid_with_all_bins_din	100.0%	100	Covered
covered/total bins:	4	4	
missing/total bins:	0	4	
% Hit:	100.0%	100	
bin <possible_values,rx_valid_high>	28471	1	Covered
bin <write_address_then_write_data,rx_valid_high>	5453	1	Covered
bin <read_address_then_read_data,rx_valid_high>	5448	1	Covered
bin <wn_address_then_wn_data_rd_address_then_rd_data,rx_valid_high>	994	1	Covered
Cross rd_data_with_tx_valid_high	100.0%	100	Covered
covered/total bins:	3	3	
missing/total bins:	0	3	
% Hit:	100.0%	100	
bin <possible_values,tx_valid_high>	6807	1	Covered
bin <read_address_then_read_data,tx_valid_high>	5452	1	Covered
bin <wn_address_then_wn_data_rd_address_then_rd_data,tx_valid_high>	997	1	Covered
ignore_bin write_address_then_write_data_with_tx_valid_high	0		ZERO
TOTAL COVERGROUP COVERAGE: 100.0% COVERGROUP TYPES: 1			

Verification Plan:

Label	Description	Stimulus Generation	Functional Coverage	Functionality Check
RAM_1	if rst_n =0 dout,tx_valid,Rd_Addr & Wr_Addr will be equal to zero	rst is made one time first by task then randomized under constraint to be deasserted by 95% and asserted 5%	No Functional Coverage for this point	Output Checked against dout_ref (reference model) compared to that in the RAM and also by assertion
RAM_2	if din [9:8] din =11 then should be tx_valid =1 only in this case	No Randomization for tx_valid as it is output	Covered by cross coverage between 2 coverpoint Transaction_ordering_cp & tx_valid_high_cp bins possible_values={2'b00,2'b01,2'b10,2'b11}; bins write_address_then_write_data=(2'b00 => 2'b01); bins read_address_then_read_data=(2'b10 => 2'b11); bins wr_address_then_wr_data_rd_address_then_rd_data= (2'b00=>2'b01=>2'b10=>2'b11); with bins tx_valid_high={1}; and ignore bins write_address_then_write_data_with_tx_valid_high= bins of (Transaction_ordering_cp.write_address_then_write_data);	Output Checked against dout_ref (reference model) compared to that in the RAM and also by assertion
RAM_3	rx_valid should be equal =1 to read & write	Randomized under constraint to be asserted 95% and deasserted 5%	Covered by cross coverage between 2 coverpoint Transaction_ordering_cp & rx_valid_high_cp bins possible_values={2'b00,2'b01,2'b10,2'b11}; bins write_address_then_write_data=(2'b00 => 2'b01); bins read_address_then_read_data=(2'b10 => 2'b11); bins wr_address_then_wr_data_rd_address_then_rd_data= (2'b00=>2'b01=>2'b10=>2'b11); with bins rx_valid_high={1};	Output Checked against dout_ref (reference model) compared to that in the RAM and also by assertion
RAM_4	din	Randomized with constraints write only ==>every Write Address operation shall always be followed by either Write Address or Write Data operation. Read only ==>every Read Address operation shall always be followed by Read Data. After a Read Data operation shall always be followed by Read Address. Read & write==> •Every Write Address operation shall always be followed by either Write Address or Write Data operation. • After a Write Data, the next operation shall be chosen with the following probability distribution: 60% → Read Address & 40% → Write Address. • Every Read Address operation shall always be followed by Read Data operation. • After a Read Data, the next operation shall be chosen with the following probability distribution: 60% → Write Address & 40% → Read Address.	covered by coverpoint Transaction_ordering_cp with bins bins possible_values={2'b00,2'b01,2'b10,2'b11}; bins write_address_then_write_data=(2'b00 => 2'b01); bins read_address_then_read_data=(2'b10 => 2'b11); bins wr_address_then_wr_data_rd_address_then_rd_data= (2'b00=>2'b01=>2'b10=>2'b11);	Output Checked against dout_ref (reference model) compared to that in the RAM and also by assertion